SUPPLEMENTARY MATERIAL: GOOD SPLITS ARE HARD TO FIND: UNDERSTANDING PERFORMANCE–ENERGY TRADE-OFFS IN HETEROGENEOUS EDGE INFERENCE

Kevin Mika [1] **Jens Hagemeyer** [1] **Ulrich Rückert** [1]

ABSTRACT

This supplement documents the deployment comparison, its quality and artifact checks, fixed proxy calibration, and controlled scheduling experiments. It includes complete shortlist curves and the budgets needed to find the best measured Native cuts. The separate graph-score and independent stage-cost studies retain their own runtime references. Full references, runtime settings, and reproduction instructions accompany the main findings.

Table 1. Deployment-comparison scope. A split measured with two runners contributes two configurations.

Quantity	Count
Models / physical setups	7 / 3
Native splits / Generic splits	204 / 204
Full alternatives	42
Deployment configurations	450
Paired Native/Generic cuts	204
Model/setup groups	21

A DATASETS AND EXPERIMENTAL SCOPE

The deployment comparison contains 204 Native splits, 204 Generic counterparts, and 42 Full alternatives. Each of the three physical setups provides an accelerator Full and a TensorRT Full for each model. All 450 configurations have throughput, energy, and numerical dataset-quality observations (Mika, 2026).

Compiler feasibility uses a broader candidate set. Of 420 accelerator–GPU candidates, 359 complete Generic evaluation, 37 fail to build, and 24 are excluded before compilation. Its 588-row source inventory also includes GPU-only controls and Full workload records. It is not the denominator of the paired deployment comparison. Native support restricts the paired subset to 17 MobileNet, 20 RegNet, and 19 ResNet cuts per setup, plus three cuts for each detector. Twelve additional detector configurations supplement the base cut design. This remains a supported subset rather than an exhaustive graph survey.

The graph-score comparison holds its Native reference measurements fixed while evaluating all six selectors. Those runtime configurations differ from the deployment comparison. The stage-cost experiment uses its own steady-state window and independently measured stages. Results are reported within each experiment rather than pooled. Exact source versions are documented in `EVIDENCE_NOTES.md` in the original research archive. File paths beginning with `analysis/` also refer to that archive. Section O describes the public reproduction files.

Across the deployment comparison, 43 configurations exceed 5% numerical relative-quality loss: 26 Native configurations, including nine accelerator Fulls, and 17 Generic configurations. The primary quality-constrained comparison retains 407 configurations: 187 Native splits, 187 Generic splits, and 33 Full alternatives. The complete 450-configuration inventory remains available for sensitivity analyses.

B GLOBAL WINNERS AND THEIR FULL REFERENCES

The main deployment comparison selects the highest measured FPS median among eligible splits and Full alternatives, separately for each model. The 5% numerical relative-quality rule retains 374 split configurations and 33 Fulls. The seven split winners and seven Full references are identical to the unrestricted selection from 450 configurations. Exact ties use ascending configuration identity. All seven selected splits use Native. Each energy value belongs to that same selected configuration.

[1]CITEC – Bielefeld University, Inspiration 1, 33619 Bielefeld, Germany. Correspondence to: Kevin Mika <kmika@techfak.uni-bielefeld.de>, Jens Hagemeyer <jhagemey@techfak.uni-bielefeld.de>.

Working draft supplement. Not submitted.

Table 2. The same split selected in the main Table 3, with two Full references. The split is not reselected by relative gain. R_h uses the fastest Full on its physical host and R_g the globally fastest Full. Consequently $R_g \leq R_h$. This comparison describes the baseline choice, not an optimization step.

Model	Selected split	FPS	Host Full	FPS	Global Full	FPS	R_h	R_g
MobileNetV3-L	DeepX/b135	1552	DeepX	1469	DeepX	1469	1.06	1.06
RegNet-X 1.6GF	H10/b123	1367	TRT	316	DeepX	538	4.33	2.54
ResNet-50	H10/b095	1100	TRT	522	DeepX	656	2.11	1.68
YOLO11l	H10/b120	53	TRT	39	DeepX	63	1.36	0.84
YOLO26m	H10/b043	110	TRT	84	TRT (H8)	85	1.31	1.30
YOLO26s	H8/b021	187	TRT	151	TRT (DeepX)	152	1.24	1.23
YOLOv7	H8/b066	113	TRT	53	DeepX	83	2.13	1.35

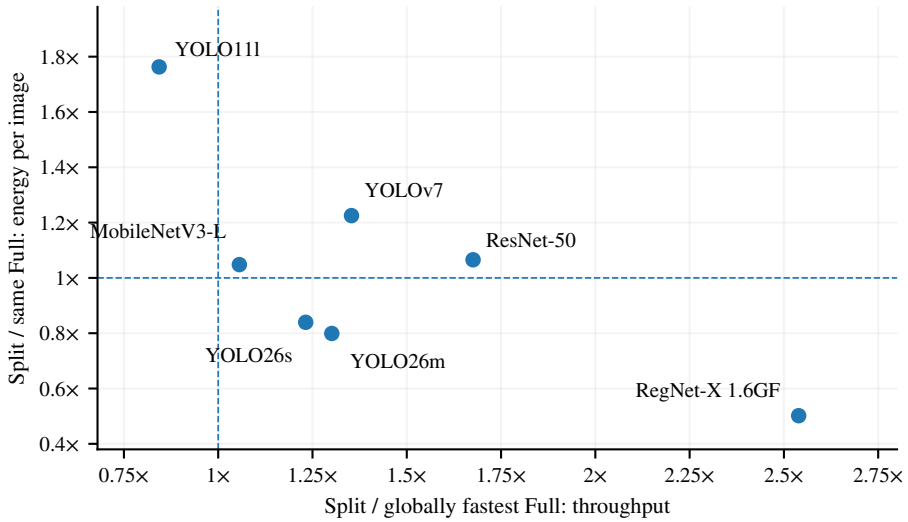


Figure 1. Throughput and energy of each model's fastest split relative to its globally fastest Full. Both axes use the same Full configuration. Right of one is faster and below one uses less energy per image. These throughput-selected points are not the full Pareto front or an energy-minimum selection.

Table 2 retains that same selected split and changes only the Full reference. Its global ratio cannot exceed its local ratio because the local Full is one of the global candidates. For RegNet the ratio changes from 4.33 to 2.54, and for YOLO11l from 1.36 to 0.84. Global selection compares deployment options when hardware choice is open. Local selection compares against available Full options on the selected split's physical host.

The mean energy of a throughput-selected Full is not necessarily the smallest Full energy. For YOLO26m, global fastest Full is TensorRT on the H8 host, at 84.76 FPS and 400.03 mJ/image. DeepX Full is slightly slower at 80.09 FPS but uses 239.32 mJ/image. The global fastest split uses 319.65 mJ/image. It improves over the FPS-selected Full's energy without being the lowest-energy overall option.

Table 3 lists the actual Full hosts and quality values. The Orin/TensorRT overview in the main Table 2 reports arithmetic means of three host-specific FPS medians and mean energies. These averages summarize the hosts. Full selection and all split gains use actual configurations.

Table 4 retains the absolute energy values behind the percentage changes in the main Table 3. FPS and energy refer to separate measurements of the same selected configurations.

Table 5 gives their observed repeat ranges and those of the two additional YOLOv7 Full references. The source projection retains the exact values and their measurement-generation bindings. MobileNet's split and Full throughput ranges overlap, while their energy ranges remain separate. These ranges describe the recorded runs and do not account for selection of the maximum or systematic measurement uncertainty.

Table 6 ranks splits by relative gain. It can choose a different split from the absolute-FPS ranking. For example, H10 MobileNet b078 has the largest local ratio, but DeepX b135 has the highest absolute split FPS. The former has 8.96% numerical relative Top1 loss. The ranking therefore

Table 3. Full alternatives on all three hosts. q is Top1 accuracy for classifiers and COCO $AP_{50:95}$ for detectors, both displayed in percent.

Model	Host	Accel. FPS	mJ/image	q (%)	TRT FPS	mJ/image	q (%)
MobileNetV3-L	H8	118	126.75	60.62	749	29.16	73.64
MobileNetV3-L	H10	210	74.96	59.42	710	27.63	73.64
MobileNetV3-L	DeepX	1469	12.01	71.72	694	27.76	73.60
RegNet-X 1.6GF	H8	362	43.24	78.76	314	77.22	78.92
RegNet-X 1.6GF	H10	259	61.79	78.84	316	74.25	78.94
RegNet-X 1.6GF	DeepX	538	31.54	78.70	333	71.81	78.94
ResNet-50	H8	122	132.45	80.32	523	56.90	80.50
ResNet-50	H10	184	89.54	80.60	522	55.17	80.50
ResNet-50	DeepX	656	27.65	80.54	517	53.97	80.50
YOLO11l	H8	11	1426.62	44.15	39	680.33	46.66
YOLO11l	H10	11	1659.47	45.31	39	653.47	46.66
YOLO11l	DeepX	63	315.23	45.67	38	675.47	46.68
YOLO26m	H8	18	935.03	39.65	85	400.03	44.82
YOLO26m	H10	14	1290.53	40.55	84	390.25	44.85
YOLO26m	DeepX	80	239.32	44.27	83	389.08	44.86
YOLO26s	H8	23	701.73	32.33	151	197.98	40.09
YOLO26s	H10	15	1108.70	34.17	150	192.20	40.09
YOLO26s	DeepX	105	176.81	39.39	152	187.80	40.11
YOLOv7	H8	32	549.84	40.78	53	598.41	46.02
YOLOv7	H10	33	601.83	41.52	53	583.57	46.02
YOLOv7	DeepX	83	269.44	45.86	48	613.18	46.04

Table 4. Absolute calibrated system energy for the same global throughput-selected Full and Native split configurations as the main Table 3. These are actual configurations, not averages across hosts. Negative energy change means less split energy per image.

Model	Full	mJ/image	Split	mJ/image	Energy change (%)
MobileNetV3-L	DeepX	12.01	DeepX/b135	12.59	+4.8
RegNet-X 1.6GF	DeepX	31.54	H10/b123	15.83	-49.8
ResNet-50	DeepX	27.65	H10/b095	29.46	+6.6
YOLO11l	DeepX	315.23	H10/b120	555.75	+76.3
YOLO26m	TRT (H8)	400.03	H10/b043	319.65	-20.1
YOLO26s	TRT (DeepX)	187.80	H8/b021	157.65	-16.1
YOLOv7	DeepX	269.44	H8/b066	330.14	+22.5

describes throughput, not equal-quality performance.

C Dataset Quality and Numerical Checks

Table 7 gives the numerical dataset quality of the global Full and split winners. Classification Top1 and detection $AP_{50:95}$ are different metrics. The relative loss is $(q_{\text{reference}} - q)/q_{\text{reference}}$, while percentage-point difference is $100(q - q_{\text{reference}})$. Those quantities must not be interchanged. All seven global split winners meet the numerical 5% point-estimate rule and remain the winners when that rule is applied before selection. Meeting that threshold does not establish equivalent accuracy.

For 53 Native DeepX classification records, the numerical evaluation and the cached relative-loss label disagree. The paper uses the evaluation values for accuracy and reports this discrepancy rather than treating the cached labels as new measurements. The data retain both values and their sources (Mika, 2026).

The quality protocol uses 5,000 examples and 1,000 paired-bootstrap replicates. Its confidence intervals compare each deployment with its CPU reference. They are not direct split-minus-Full intervals or intervals for FPS maxima. The retained YOLOv7 case summaries contain no per-image predictions or bootstrap samples from which a direct split-minus-Full interval could be recomputed. Parity checks on a timing input or a mixed finite sequence test the corresponding runtime and samples, not the full dataset again. Integer equality, finite values, matching shapes and dtypes, and the recorded floating-point tolerances are retained in the acquisition evidence.

D Runtime and Measurement Boundaries

Table 8 lists available platform and runtime records. The scope column identifies which configurations each record covers. An x86 controller coordinates the three Orin NX hosts. All 204 splits execute an accelerator prefix followed

Table 5. Observed minimum–maximum over three repetitions for the global winners in the main Table 3 and the additional Full references in the YOLOv7 case. FPS comes from performance runs and energy from separate runs. These are ranges, not confidence intervals. FPS endpoints are rounded to integers.

Deployment	FPS	mJ/image
MobileNetV3-L / DX Full	1,458–1,470	11.98–12.05
MobileNetV3-L / DX/b135	1,441–1,570	12.58–12.59
RegNet-X / DX Full	538–541	31.48–31.63
RegNet-X / H10/b123	1,103–1,386	15.78–15.88
ResNet-50 / DX Full	655–659	27.64–27.66
ResNet-50 / H10/b095	1,074–1,114	29.42–29.50
YOLO11l / DX Full	63–63	314.56–315.72
YOLO11l / H10/b120	53–53	537.63–567.69
YOLO26m / TRT (H8)	85–85	399.41–400.41
YOLO26m / H10/b043	110–111	319.55–319.76
YOLO26s / TRT (DX)	152–152	186.79–188.63
YOLO26s / H8/b021	187–188	—
YOLOv7 / DX Full	83–84	268.19–270.38
YOLOv7 / H8/b066	112–113	329.43–330.50
Additional YOLOv7 references on H8		
YOLOv7 / H8 Full	32–32	549.54–550.25
YOLOv7 / TRT (H8)	53–53	596.63–599.40

The YOLO26s split energy mean is available, but its individual repetitions are absent from the retained source extract. The dash denotes this missing range.

by a TensorRT suffix.

The quality records identify an ImageNet validation subset and a COCO 2017 validation set of 5,000 images each. Dataset hashes agree across all 354 classification configurations and, separately, all 96 detector configurations. These identities bind the comparisons to the same images. The compact records do not specify the subset selection algorithm.

Explicit preprocessing objects are retained for 298 classifier and 87 detector configurations. The classifier objects specify RGB input and direct bilinear resizing to 224×224. Detector objects specify RGB input and centered bilinear letterboxing to 640×640, with padding value 114. The remaining configurations are linked through the completed audit and source records. These contracts describe the prepared RGB uint8 image stage. They do not specify every backend-specific scaling and normalization step. The projection under `analysis/r20_evidence/` retains each observed object and its source location.

The bridge inventory contains 124 split records labelled `float32_layout_fp16` and 80 labelled `uint8_dequant_fp16`. These names describe boundary representation and bridge conversion, rather than the precision of every accelerator operator. Full implementations retain their own precision contracts.

The nine pathway projections comprise MobileNetV3/DX b001 and H8 b027. RegNet/H10 b052 and b123. ResNet/H10 b002 and b095. and YOLOv7/H10 b066 and H8 b044/b066. They are retained in `analysis/evidence/inputs/ranking_path_evidence.csv`. The H8 b066 projection additionally records a postprocessing queue of four. The timing workload that repeats a prepared feed is separate from the 5,000-example task-quality evaluation. A universal batch size, input schedule, queue depth, or affinity policy for all 204 configurations is not inferred from these nine records.

Generic timing begins with prepared input before the first partition and ends at the last completed task, including materialization, task completion and fill/drain. Native records preserve a worker-ready start boundary. Earlier application preprocessing is outside the Generic timing scope. The resulting pipeline throughput is not per-request or camera-to-detection latency.

The YOLOv7 configuration report is archived as `analysis/reported_update/RESULT_REPORT.md`. Its separate Python-loader probe records CUDART 12.9.79 on H10 and 12.6.68 on H8/DX, with driver API 12.6 on all three. This probe does not observe libraries loaded by earlier measurement processes or by the H8 C++ shared object. A separate H8 runner-import check reports NumPy 1.26.4, TensorRT 10.3 and HailoRT 4.20. it does not establish the software stack of every setup. Load samples at approximately 1 Hz cannot resolve brief intervening clock or thermal events, and the report supplies no EMC measurement.

The compact deployment inputs do not fully reconstruct checkpoint and dataset-manifest identities, exporter/opset versions, all device/compiler/runtime versions, or contemporaneous power modes, clock/thermal traces and affinity settings. The analysis environment in the reproduction metadata is distinct from device software. These missing fields limit exact reacquisition and causal attribution of cross-host differences. they do not change the recorded per-host split/Full denominators. The DeepX Native policy is uniform across the seven models and all measured supported cuts: batch one, one engine per process, actual asynchronous submission, and at most four outstanding requests. Four separate output slots preserve the lifetime and identity of each request. Split execution retains the required owned boundary copy, a FIFO of depth three, and a serial TensorRT consumer including H2D, GPU execution, D2H/synchronization, and task completion. Full uses the same A4 execution policy without an artificial partition boundary. A4 is not selected per model from whichever mode is fastest.

Generic and the Hailo Native paths use their respective implementations and concurrency settings. In particular, H8 YOLOv7 b066 uses a three-stage arrangement, whereas

Table 6. Two Native splits per model with the largest relative throughput gain over the fastest Full on the same host. Unlike the main Table 3, selection maximizes relative gain rather than absolute FPS. $G = e_{\mathrm{Full}}/e_{\mathrm{split}}$ uses that same Full. Quality is reported separately.

Model	Rank	Setup/cut	Full FPS	Split FPS	R_h	mJ/image	G
MobileNetV3-L	1	H10/b078	710	1172	1.65	20.09	1.38
MobileNetV3-L	2	H10/b074	710	1143	1.61	19.94	1.39
RegNet-X 1.6GF	1	H10/b123	316	1367	4.33	15.83	4.69
RegNet-X 1.6GF	2	H10/b132	316	1175	3.72	16.95	4.38
ResNet-50	1	H10/b095	522	1100	2.11	29.46	1.87
ResNet-50	2	H10/b088	522	1060	2.03	30.57	1.80
YOLO11l	1	H10/b120	39	53	1.36	555.75	1.18
YOLO11l	2	H10/b062	39	51	1.31	558.64	1.17
YOLO26m	1	H10/b043	84	110	1.31	319.65	1.22
YOLO26m	2	H10/b038	84	108	1.29	323.19	1.21
YOLO26s	1	H8/b021	151	187	1.24	157.65	1.26
YOLO26s	2	H10/b021	150	182	1.21	168.51	1.14
YOLOv7	1	H8/b066	53	113	2.13	330.14	1.81
YOLOv7	2	H10/b044	53	70	1.32	455.36	1.28

Table 7. Dataset quality of the globally fastest Full and split configurations. Relative loss is measured against the CPU reference. Negative loss means a higher point estimate, not a significant improvement. Throughput alone selects these configurations.

Model	Metric	Split	q (%)	Loss (%)	Full	q (%)	Loss (%)
MobileNetV3-L	Top1	DeepX/b135	72.02	2.28	DeepX	71.72	2.69
RegNet-X 1.6GF	Top1	H10/b123	79.16	-0.28	DeepX	78.70	0.30
ResNet-50	Top1	H10/b095	80.16	0.35	DeepX	80.54	-0.12
YOLO11l	AP	H10/b120	45.55	2.42	DeepX	45.67	2.17
YOLO26m	AP	H10/b043	43.66	2.75	TRT (H8)	44.82	0.18
YOLO26s	AP	H8/b021	39.18	2.26	TRT (DeepX)	40.11	-0.05
YOLOv7	AP	H8/b066	45.74	0.64	DeepX	45.86	0.38

b009/b044 combine TensorRT and completion in one consumer. The comparison therefore evaluates the available implementations. It does not hold language, physical host, or pipeline organization constant across all accelerators.

The timed endpoint is completed classification or detection. Classification includes synchronized outputs and Top1/Top5 processing. Detection includes the decoding, NMS or integrated output, and materialization required by the compiled model. YOLO26 integrated paths use their valid graph endpoint, while raw-head paths retain the required decoding and NMS. The independent correctness oracle for the 36 Native detector splits is outside timing, but this does not remove required task completion from their measured path. Generic detector splits execute their configured decoding and NMS. DeepX Fulls use host postprocessing or integrated accelerator completion followed by normalization. These implementation differences remain explicit in the per-configuration records.

The detector comparison contract specifies original-image $xyxy$ coordinates in pixels, a probability score, and an integer class index. It uses a score threshold of 0.25 and at most 300 detections. Where NMS is required, the policy is class-aware with an IoU threshold of 0.45. The integrated YOLO26 paths retain their graph-defined completion behavior. Table 9 separates these paths from raw-head execution.

All 96 detector configurations have a runtime comparison contract, with the same contract hash within each model across Generic, Native, and Full. The source projection also contains 65 explicit quality comparison objects and 52 explicit performance comparison objects. Every available object matches its runtime counterpart. The completed artifact audit supplies the remaining source bindings. Matching contracts define the required result, while dataset evaluation measures numerical quality.

The archived `analysis/r20_evidence/endpoint_contracts.csv` retains the original endpoint checks for 20 central configurations, including all fourteen global winner/reference configurations. These checks confirm required postprocessing and materialization inside timing. The detector projection records the contracts for all 96 configurations and their source hashes. Supplement Section G describes the wider 450-configuration audit and the separate checks with different inputs.

The deployment measurements use three independent processes with 1,000 measured tasks each. The inspected path records specify 100 warmup tasks, completed and drained before timing. The timer includes fill, necessary per-task copies, task completion, and drain. Repeated prepared in-

Table 8. Recorded setup and workload metadata. The YOLOv7 platform observations describe its setup comparison. Other deployments retain their own recorded settings.

Item	Recorded setting	Evidence scope
Performance workload	Three repetitions and 1,000 completed tasks per repetition	204 split and 42 Full configurations. YOLOv7 combines 27 new split and 18 preserved Full repetitions
Batch and Generic queues	Batch 1. two workers. queue depth 2. 100 warmups. one exactly prepared feed repeated	Nine targeted pathway records, not all 204 splits
Native queues	Queue depth 3. 100 warmups. H10 paths additionally record 8 prefix requests in flight	The same nine targeted pathway records. runtime organization remains case-specific
Nominal spatial inputs	ImageNet validation 224×224 and COCO 2017 validation 640×640	Dataset identities and available preprocessing contracts, described below
Orin configuration	NX 16 GB. L4T 36.4.7. matching active `MAXN_SUPER` definition	YOLOv7 setup comparison
Clock and link observations	CPU 1984 MHz. GPU 1173 MHz. no observed OC-counter increments. accelerator PCIe snapshots 8.0 GT/s $\times 4$	Approximately 1-Hz YOLOv7 load samples and separate link snapshots. EMC unobserved
TensorRT Full build	Same model, FP16 option, static FLOAT input/output shapes and workspace setting	YOLOv7 Full comparison. serialized engines and internal tactics need not be identical
DC energy boundary	Calibrated full-system input energy. no idle subtraction in the primary comparison	231 cases / 693 repetitions over six models. calibration detail in Section N

Table 9. Required task outputs and completion work. Generic, Native, and Full use the same model-specific comparison contract. The compiled graph determines whether decoding and suppression occur in the model or in host code. Required output processing and materialization precede task completion.

Model or output path	Completed result	Required processing
MobileNet, RegNet, ResNet	Synchronized classifier outputs and ordered Top1/Top5	Output materialization and Top1/Top5 processing.
YOLO11l and YOLOv7	Original-image boxes, probability scores, and class indices	Model-specific decoding and class-aware NMS where not already completed by the model, followed by materialization.
YOLO26m/s, raw-head paths	The same detection record format	Decode raw heads and apply the required NMS. This includes Hailo Full paths.
YOLO26m/s, integrated outputs	The same detection record format	Filter and normalize integrated detections, undo letterboxing, and materialize outputs. No additional NMS pass is applied to valid integrated outputs.

puts define a controlled microbenchmark. Their throughput is not camera-to-detection latency, and $1/\mathrm{FPS}$ is not a per-request latency under concurrent execution.

Energy acquisition requests 60 s of work and completes admitted tasks before reporting the command end. The primary metric is the mean of three calibrated E_r/N_r values. Total joules, durations, counts, individual repeats, and alternative normalizations remain in the source records. Performance FPS and energy are independent measurements of the same execution configuration.

Device state. Clock, affinity, and temperature observations are available per run where recorded. Missing state measurements limit exact reacquisition and attribution of cross-host differences. A total runtime or number of pending requests does not identify NPU occupancy, thermal throttling, PCIe traffic, or scheduler overhead. The separate synchronization diagnostic and its temperature criteria are documented in `EVIDENCE_NOTES.md`, not used as additional deployment measurements.

Full and split context. DeepX Full uses less energy than TensorRT Full on the same host for all seven models in the main dataset. This does not imply that it has the highest throughput for every model. MobileNet/H8 b027 illustrates a different baseline issue: its Native throughput is 8.40 times the accelerator Full rate, but only 1.33 times the faster local TensorRT Full rate. The corresponding energy gain against that TensorRT Full is 1.27. Its Top1 recovery is reported with the compiler diagnostics below. Together these observations concern the complete compiled recipe, not quantization alone.

Table 10. YOLOv7 deployment comparison on three cuts per setup. G and N denote Generic and Native. The final column reports Native energy.

Host	Cut	G FPS	N FPS	N mJ/image
H8	b009	41	45	599.65
H8	b044	45	50	515.66
H8	b066	47	113	330.14
H10	b009	29	51	587.48
H10	b044	47	70	455.36
H10	b066	28	53	514.55
DeepX	b009	37	52	594.99
DeepX	b044	50	64	469.30
DeepX	b066	32	63	481.86

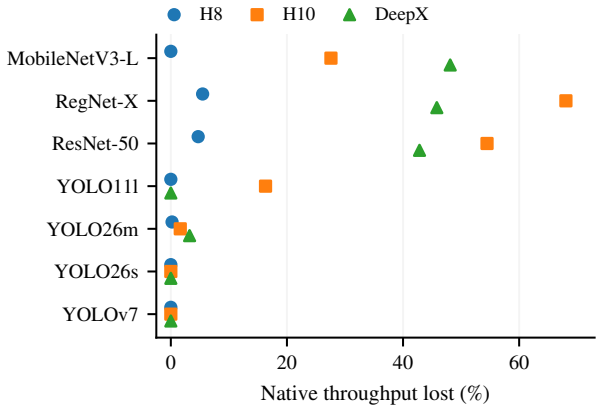


Figure 2. Generic-to-Native selection before quality filtering, using all 204 paired cuts. The equally weighted mean loss across 21 groups is 15.16%. Figure 3 shows the corresponding quality-filtered result.

E Generic Selection and Energy Choices

Across these 204 pairs, Native is faster in 186 and uses less energy in 196. All 186 faster pairs also use less energy. The pooled median Native/Generic FPS ratio is 1.60. The 204 pairs are joined by model, normalized setup, and cut. Within each group, the Generic recommendation maximizes its measured FPS, then the selected cut is evaluated by its Native FPS. This yields nine exact winner agreements across 21 groups and 15.16% group-mean loss. Only one classification group agrees, compared with eight of twelve detector groups. A Top3 Generic shortlist includes the Native winner in two of nine classifier groups. Every detector group has exactly three candidates, making its Top3 coverage automatic.

The DeepX classification results illustrate target dependence. Generic chooses b056, b052, and b002 for MobileNet, RegNet, and ResNet. Native A4 instead prefers b135, b109, and b104. The corresponding losses are 48.12%, 45.82%, and 42.84%. These recommendations are evaluated against the target Native implementation. The experiment does not isolate Generic overhead under identical schedules.

Table 10 gives the complete YOLOv7 deployment comparison. Both runners select b066 on H8 and b044 on H10 and DeepX. All nine Native entries have matching energy observations.

The fastest and energy-minimizing Native split differ in five of 21 groups: MobileNet/H10, RegNet/DeepX, ResNet/DeepX, ResNet/H8, and YOLO26m/H8. For ResNet/H8, b038 maximizes performance throughput and b060 minimizes energy. The separate energy-window study below uses another rate definition and measurement set. The empirical Pareto front contains 33 nondominated identities across the 21 model/setup groups, retaining both Full alternatives and both runners. This empirical frontier is not a claim of equal-quality or equal-work dominance.

F Retrospective Quality and Budget Analyses

The following analyses use the deployment comparison's 408 split and 42 Full records. They add no measurements. Eligibility is calculated as $(q_{\mathrm{ref}} - q)/q_{\mathrm{ref}} \le \epsilon$ from the numerical evaluation values. The recalculated losses match the stored numerical losses. Cached policy labels are not used. The artifact audit below establishes the bindings used for inclusion. Quality filtering alone does not establish identical schedules, numerical accuracy, or software implementations.

Table 12 filters both Native splits and Full alternatives, then reselects the fastest and lowest-energy Full on each host. At $\epsilon = 0.05$, 187 Native splits and 33 Full alternatives remain. All seven global split and Full winners are unchanged. Local optima can change. MobileNet/H10 b078 has 8.96% relative quality loss. The admissible optimum is b074 at 1,142.90 FPS. Generic still selects b027, whose target-throughput loss changes from 27.58% to 25.75%.

The 5% Native and Generic quality filters retain exactly the same cuts. Requiring both runners to pass therefore gives the same results as requiring Native quality alone. This is an observation of this dataset, not an assumed equality of their outputs. All quality values are already known when the sensitivity analysis is performed. It does not provide a free prospective quality screen.

For each model/setup group, the Generic shortlist contains the first k admissible cuts in descending Generic FPS order, with configuration identity breaking ties. The retained deployment is the fastest measured Native member. Uniform sampling selects k distinct candidates without replacement.

Table 11. Generic-to-Native selection in the deployment comparison. Each row is one model/setup group. L_R is the Native throughput lost by choosing the Generic winner. Choices use unrounded FPS.

Model	Host	n	Generic choice	Native best	Selected FPS	Best FPS	L_R (%)
MobileNetV3-L	H8	17	b027	b027	993	993	0.00
MobileNetV3-L	H10	17	b027	b078	849	1172	27.58
MobileNetV3-L	DeepX	17	b056	b135	805	1552	48.12
RegNet-X 1.6GF	H8	20	b066	b073	584	618	5.48
RegNet-X 1.6GF	H10	20	b052	b123	437	1367	68.05
RegNet-X 1.6GF	DeepX	20	b052	b109	395	729	45.82
ResNet-50	H8	19	b002	b038	554	581	4.71
ResNet-50	H10	19	b002	b095	501	1100	54.47
ResNet-50	DeepX	19	b002	b104	447	782	42.84
YOLO11l	H8	3	b120	b120	44	44	0.00
YOLO11l	H10	3	b003	b120	45	53	16.31
YOLO11l	DeepX	3	b062	b062	50	50	0.00
YOLO26m	H8	3	b043	b038	79	79	0.21
YOLO26m	H10	3	b038	b043	108	110	1.63
YOLO26m	DeepX	3	b038	b043	95	98	3.23
YOLO26s	H8	3	b021	b021	187	187	0.00
YOLO26s	H10	3	b021	b021	182	182	0.00
YOLO26s	DeepX	3	b021	b021	163	163	0.00
YOLOv7	H8	3	b066	b066	113	113	0.00
YOLOv7	H10	3	b044	b044	70	70	0.00
YOLOv7	DeepX	3	b044	b044	64	64	0.00

Table 12. Quality-constrained Native advantages on the same host. Each row filters both Native splits and Full alternatives using the numeric quality loss against the CPU reference, then reselects the fastest and minimum-energy Full. Faster and Lower energy compare against the same fastest Full. Both requires both advantages. Below min. Full compares energy against the lowest-energy admitted Full. These are retrospective point-estimate comparisons.

Eligibility	Native cuts	Fulls	Faster	Lower energy	Both	Below min. Full
All measured	204	42	105	77	65	72
$\leq 1\%$ loss	114	28	58	40	32	37
$\leq 2\%$ loss	147	30	88	61	52	56
$\leq 5\%$ loss	187	33	102	74	62	69

The original retrospective table uses $k \in \{1, 3, 5\}$, capped at the available candidate count. The fixed calibration comparison below instead marks budgets larger than a group as unavailable, so it reports no detector $k = 5$ result. The budget counts Native throughput measurements only. It excludes Generic acquisition, compilation, and dataset-quality evaluation.

Let $r_1 \leq \cdots \leq r_n$ be the Native rates in an eligible group. The expected best rate of a uniformly chosen subset of size k is

$$\mathbb{E}[R_k] = \sum_{i=k}^{n} r_i \frac{\binom{i-1}{k-1}}{\binom{n}{k}}. \tag{1}$$

The expected loss is $100(1 - \mathbb{E}[R_k]/r_n)$. The probabilities are exact and use no Monte Carlo trials. The analysis independently enumerates all subsets for these groups to check the result. Group means give equal weight to each of nine classification and twelve detection groups. Expected hit counts for random selection are sums of the probabilities of including a throughput-optimal candidate.

Across all measured cuts, Generic's mean loss is 15.16%, versus 23.78% for one uniformly chosen cut. After the 5% filter, these values are 15.08% and 21.89%. The aggregate masks a family difference. For eligible classifiers, Generic Top3 and Top5 retain losses of 23.77% and 20.45%, compared with 15.41% and 9.10% for uniformly sampled subsets of the same sizes. Eligible detectors favor Generic for one measurement. Three measurements cover their entire candidate sets. A descriptive 1% throughput-loss tolerance raises the one-choice Generic success count from nine to ten of 21 groups, both before and after quality filtering. This tolerance is not a confidence interval or a significance test.

The stored graph scores are not replayed against a different experiment's Native measurements. Matching model/setup/cut labels alone does not establish identical graph artifacts across experiments. Their original comparisons remain separate. The new boundary-bytes comparison below recomputes that fixed rule from the graphs bound to the deployment measurements. It does not import the older score table. The original identity check and quality-budget

Table 13. Finite-budget selection over the deployment-comparison observations. Generic (G) orders every available cut, after which k Native measurements select the fastest shortlisted cut. Uniform sampling (U) uses exact subset probabilities without replacement. Loss is measured against the fastest Native candidate in the same eligible set. Hits for U are expected counts, not observed trials. Budgets are capped at the number of available candidates. The quality filter uses known Native quality retrospectively. Requiring Generic quality as well gives identical sets at 5%.

Eligibility	Family	Groups	Cuts	k	G loss (%)	U loss (%)	G hits	U hits
All measured	All	21	204	1	15.16	23.78	9	4.484
All measured	All	21	204	3	10.28	7.51	14	13.453
All measured	All	21	204	5	8.87	4.43	15	14.422
All measured	Classification	9	168	1	33.01	36.71	1	0.484
All measured	Classification	9	168	3	23.99	17.51	2	1.453
All measured	Classification	9	168	5	20.69	10.34	3	2.422
All measured	Detection	12	36	1	1.78	14.09	8	4.000
All measured	Detection	12	36	3	0.00	0.00	12	12.000
All measured	Detection	12	36	5	0.00	0.00	12	12.000
$\leq 5\%$ loss	All	21	187	1	15.08	21.89	9	4.603
$\leq 5\%$ loss	All	21	187	3	10.19	6.60	14	13.808
$\leq 5\%$ loss	All	21	187	5	8.76	3.90	15	15.014
$\leq 5\%$ loss	Classification	9	151	1	32.80	32.29	1	0.603
$\leq 5\%$ loss	Classification	9	151	3	23.77	15.41	2	1.808
$\leq 5\%$ loss	Classification	9	151	5	20.45	9.10	3	3.014
$\leq 5\%$ loss	Detection	12	36	1	1.78	14.09	8	4.000
$\leq 5\%$ loss	Detection	12	36	3	0.00	0.00	12	12.000
$\leq 5\%$ loss	Detection	12	36	5	0.00	0.00	12	12.000

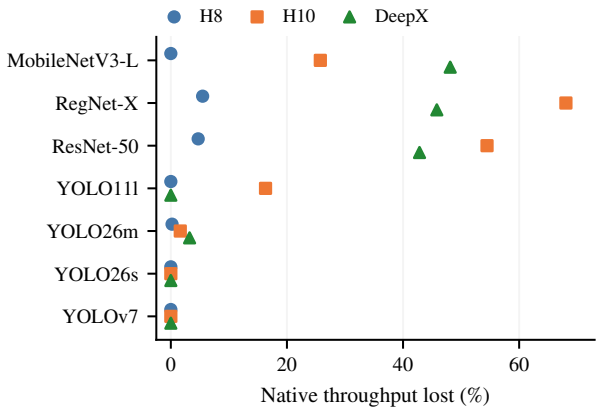


Figure 3. Generic-to-Native selection after the 5% quality filter, using 187 paired cuts. Each marker shows the Native throughput lost by choosing the Generic winner in one model/setup group. Generic finds the Native winner in nine of 21 groups, with 15.08% mean loss.

calculations remain under `analysis/review_r14_output/`.

G Artifact Audit and Eligibility

The audit checks all 450 deployment configurations, including every Full alternative. It connects dataset quality, compiled artifacts, prepared inputs, the completed-task endpoint, performance repetitions, and energy records. Configuration names alone are not treated as artifact matches. The 407 configurations retained by the 5% quality rule have the required bindings. The remaining 43 are excluded because of numerical quality, rather than a missing measurement or an administrative label. All fourteen global winner configurations remain eligible.

Resolved source restrictions. The 75 DeepX Native configurations already use the A4 execution policy described above. Their 225 performance processes and 225 energy repetitions were completed before this audit. Each energy configuration has three 60 s repetitions, the applicable calibration, and the final acquisition protocol. Earlier screening labels arose from a previous admission and quality classification. The underlying records support the measured configurations without replacing their throughput or energy values.

Six YOLO26 split aggregates retained partial status despite three valid raw repetitions each. The discrepancy arose from path-dependent identity records after files moved. The relevant artifact hashes match. Seventeen quality-receipt version discrepancies likewise do not imply changed compiled artifacts. The selected evaluation files, their hashes, the normal execution reports, and the runtime artifact chain were checked independently. Original records and labels are preserved. Corrected inclusion decisions are separate derived data.

For Hailo Full configurations, five have contemporaneous ar-

tifact hashes before and after all three repetitions. The other nine have composite production records that connect the exact performance report with its HEF, graph, and preparation. This evidence supports retaining the strong Full references. It does not reconstruct every historical software condition.

Input association. The historical throughput protocol repeats a prepared input. Distinct-input checks are available for the seven DeepX Full configurations and the selected MobileNet/DeepX b135 split. The latter test uses 32 different images. All 32 serial references and asynchronous outputs match exactly at the boundary and logits. These eight configuration-specific checks do not establish arbitrary request/output association across all 450 configurations. They add neither dataset-quality estimates nor throughput or energy observations.

Resulting comparison. Of the 187 eligible Native splits, 102 exceed their fastest eligible local Full, 74 use less energy than that same Full, and 62 satisfy both conditions. Against the independently lowest-energy Full, 69 use less energy. In the 187 paired comparisons, Native is faster than Generic in 169 cases and uses less energy in 179. The median throughput ratio is 1.602. Without quality filtering, the corresponding counts are 186 and 196 of 204, with a median ratio of 1.595. These compare the documented deployment implementations and their required task endpoints. They do not hold their scheduling policies constant.

H Fixed Calibration and Boundary-Bytes Selection

This retrospective comparison uses the same 151 eligible classifier pairs for every method. They form nine model/setup groups: three architectures on each of three setups. The formula and penalty were fixed before fitting the folds. For each setup, one complete architecture is held out and the other two supply training data. Standardization uses only training rows.

For candidate i, the model predicts

$$\widehat{\log R_{N,i}} = \log R_{G,i} + \beta_0 + \beta_1 z(x_i) + \beta_2 z\big(\log(1+b_i)\big), \tag{2}$$

where x_i is the prefix share of graph-estimated computation and b_i is the logical boundary size in bytes. The response used in training is $y_i = \log(R_{N,i}/R_{G,i})$. We minimize

$$\frac{1}{n}\sum_i (y_i - \widehat{y}_i)^2 + \beta_1^2 + \beta_2^2. \tag{3}$$

The intercept is unpenalized and the ridge penalty is fixed at one. Feature scales are population standard deviations from training rows. A constant feature receives scale one and standardized value zero. There is no feature selection, hyperparameter search, or cut-level random train/test split.

Table 14. Classifier selection loss (%) after the 5% quality filter. Nine groups receive equal weight. Random values are exact expectations. Budgets count held-out Native measurements only.

Method	$k=1$	$k=3$	$k=5$
Generic	32.804	23.768	20.448
Calibrated Generic	28.319	23.146	15.431
Uniform random	32.290	15.406	9.104
Boundary bytes	32.966	21.747	8.862

Winner coverage	$k=1$	$k=3$	$k=5$
Generic	1	2	3
Calibrated Generic	1	3	4
Uniform random	0.603	1.808	3.014
Boundary bytes	1	3	5

Coverage counts groups containing a Native optimum, out of nine. Random coverage is an expected count.

All feature graphs are bound to the measured artifacts. The boundary-bytes baseline ranks candidates by increasing b_i and is recomputed from these graphs. Generic ranks by measured Generic throughput. The calibrated method ranks by predicted Native throughput. Configuration identity resolves ties. Each method selects k candidates and retains their highest measured Native throughput. The loss is relative to the eligible Native optimum in the same group. Uniform sampling uses the exact finite-set expectation from Section F. The nine groups receive equal weight.

The calibration reduces mean classifier loss, but the improvement comes from DeepX. Averaged over the three H10 groups, it is worse than uncalibrated Generic at each of the three tabulated budgets. On H8, mean loss at $k=5$ increases from zero to 1.825%. The boundary-bytes rule has a slightly smaller mean loss than random selection at $k=5$, by 0.242 percentage points. Its performance varies strongly across groups. For ResNet/H10, its loss is 54.467%, compared with 11.666% for random selection. These results support neither uniform failure nor general superiority of simple static rules.

The 36 eligible detector candidates form twelve groups with three cuts each. Generic's one-choice mean loss is 1.782%, compared with 14.091% for uniform sampling. Three measurements exhaust every group, so the original table omits $k=5$. Neither the classifier calibration nor the recomputed classifier boundary-bytes comparison is extended to detectors.

The held-out architecture has no rows in its fold's fit. Nevertheless, all three architectures were already known when this retrospective study was designed. The nine folds are not independent samples from a population of architectures.

Table 15. Complete classifier selection losses (%) by group and budget. All four methods use the same eligible cuts. Each triplet is $k = 1/3/5$. n is the number of eligible cuts. The calibrated method holds out the entire named architecture during fitting on the indicated setup.

Model	Setup	n	Generic	Calibrated Generic	Uniform random	Boundary bytes
MobileNet	DeepX	17	48.12/44.61/44.04	15.83/4.22/0.00	39.38/22.79/14.40	0.00/0.00/0.00
MobileNet	H10	9	25.75/18.56/10.06	25.75/18.56/10.06	18.11/6.74/3.29	10.06/6.80/0.00
MobileNet	H8	8	0.00/0.00/0.00	0.00/0.00/0.00	21.37/5.25/1.39	40.18/40.18/0.00
RegNet	DeepX	20	45.82/45.49/42.56	45.82/45.49/35.32	35.51/16.69/9.40	25.20/15.11/15.11
RegNet	H10	20	68.05/65.21/65.21	68.05/65.21/62.94	54.98/34.35/23.69	14.08/0.00/0.00
RegNet	H8	20	5.48/5.48/0.00	5.48/5.48/5.48	28.72/16.38/11.19	42.62/18.96/5.48
ResNet	DeepX	19	42.84/13.25/8.12	13.25/0.00/0.00	20.43/5.42/2.01	13.25/0.00/0.00
ResNet	H10	19	54.47/21.31/14.04	75.98/69.35/25.09	44.80/20.69/11.67	75.98/69.35/54.47
ResNet	H8	19	4.71/0.00/0.00	4.71/0.00/0.00	27.32/10.35/4.92	75.33/45.33/4.71

Exact random expectations are not measurement confidence intervals. The budget k counts Native measurements in the held-out group only. Training and Generic acquisition use additional measurements. Across all folds, 151 distinct Native and 151 distinct Generic configurations are reused. Fold counts must not be summed as independent acquisitions. Compilation, quality evaluation, and total search time were not measured, so this analysis makes no total-cost saving claim.

H.1 Complete budgets and successful Native selection

We extend the shortlist analysis from $k = 1, 3, 5$ to every integer budget up to the largest group. At each maximum budget k, a group with n candidates uses $\min(k, n)$ trials. This keeps the same nine classifier groups or twelve detector groups in every mean. The output records the minimum, maximum, mean, and total number of actual trials.

Success means that the shortlist contains an exact measured Native winner, or a cut with at most 1% or 5% less throughput than that winner. Exact success uses the unrounded recorded FPS values and includes tied winners. These throughput tolerances are separate from the CPU-relative 5% task-quality filter. The first successful rank is the smallest shortlist that meets the chosen criterion.

On the 151 quality-admissible classifier cuts, the boundary-size rule recovers eight of nine exact winners at $k = 12$ and all nine at $k = 14$. All nine groups contain a cut within 1% of their optimum at $k = 13$, or within 5% at $k = 11$ (Figure 4). Generic and calibrated Generic each need $k = 18$ for all nine exact winners. A maximum budget of 14 uses 115 trials in total because the smaller groups are already exhausted. A budget of 11 uses 94 trials, compared with 151 for complete evaluation. These counts concern Native tests among the already quality-evaluated candidates.

The separate graph-score measurement set needs 13 boundary-size trials for all nine exact classifier winners, compared with 14 in the deployment comparison. This difference follows from its Native reference rates. For example, ResNet/H8 prefers b031 in the graph-score set and b038 in the deployment data. Their boundary-size ranks are 13 and 14, respectively. Section J gives the complete graph-rule comparison.

Observed coverage and random success. For a fixed ranking, the fraction of successful groups is observed coverage on these experiments. Eight of nine groups correspond to 88.9%. This is not a success probability for an unseen model. For uniform sampling without replacement, the probability of finding at least one acceptable cut is

$$P_k = 1 - \frac{\binom{n-m}{k}}{\binom{n}{k}}, \tag{4}$$

where m of the n candidates meet the chosen throughput criterion and $k \le n$. The numerator is zero if $k > n - m$. For a unique exact winner, $P_k = k/n$. We average these probabilities equally across the fixed groups. This quantity differs from expected throughput loss.

Random selection first reaches 95% mean probability of finding an exact winner at a maximum budget of 19 in both comparisons in Table 16. In the quality-filtered deployment comparison, the corresponding budgets are 17 for the 1% throughput tolerance and 16 for the 5% tolerance. These probabilities concern the available candidates. The retrospective first-hit ranks do not provide a stopping rule for a new search whose optimum is unknown.

Coverage and reproduction. Both measurement sets contain 204 paired cuts, including 168 classifier and 36 detector cuts. The deployment quality filter retains 151 classifier cuts and all 36 detector cuts. Verified boundary-byte features cover all 168 deployment classifier cuts, while frozen calibrated predictions cover only the 151 eligible ones. The analysis includes both the unfiltered and quality-filtered views where the required inputs exist. It does not transfer

Table 16. Minimum maximum Native-trial budget per classifier group. Each measurement set contains nine groups. Columns require an exact optimum in eight or nine groups, or a cut within 1% or 5% of the Native optimum in all nine. Budgets are capped at each group's candidate count. Throughput tolerances are separate from the deployment quality filter.

Measurement set	Selector	Exact, 8/9	Exact, 9/9	$\leq 1\%$, 9/9	$\leq 5\%$, 9/9
Graph-score comparison	Bytes	12	13	13	11
	Weighted	18	19	19	19
	Fit-H10	15	19	19	19
	Cycle	18	18	18	18
	Stream	18	18	18	18
	Generic	15	18	18	18
Deployment comparison	Bytes	12	14	13	11
	Generic	17	18	18	18
	Calibrated	13	18	18	18

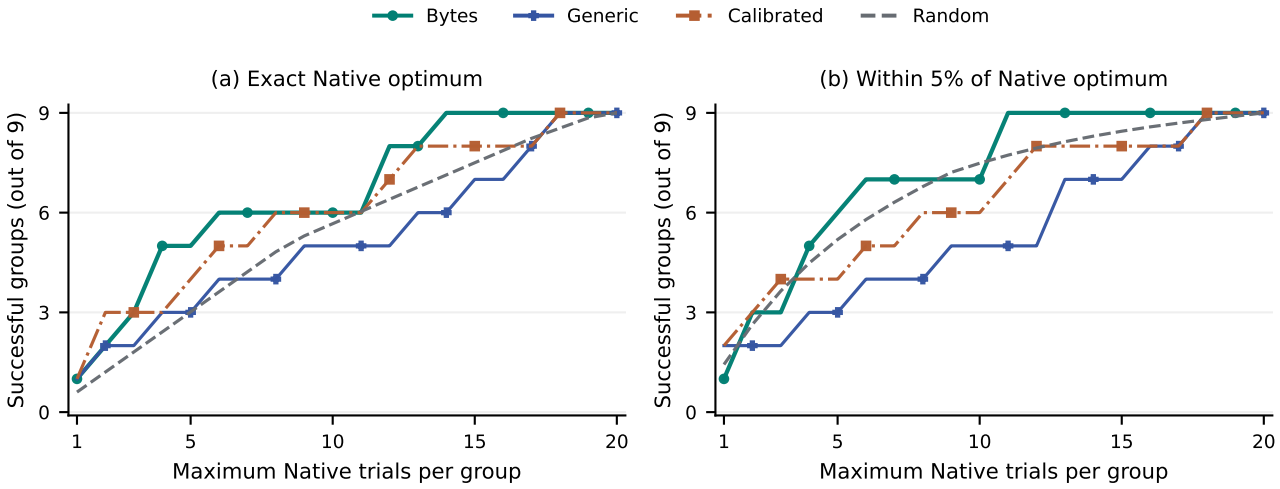


Figure 4. Successful Native selection on the 151 quality-admissible classifier cuts. Left: the shortlist contains an exact Native winner. Right: its best Native cut loses at most 5% throughput. Deterministic curves count successful groups among the same nine model/setup groups. Random curves average the exact success probabilities for those groups. The candidate quality filter and the throughput tolerance are distinct.

graph scores to the 36 deployment detector cuts without verified feature bindings.

The exact-winner curves reproduce the existing shortlist winner counts at every budget. They are monotone and reach one when every candidate is tested. The recorded inputs, complete curves, and generators are included in the public reproduction files described in Section O.

I Controlled Scheduling Experiments

These experiments test whether the relative throughput of two cuts changes when concurrency is restricted within each runtime chain. They use RegNet/H10 b052 and b123, and ResNet/H10 b002 and b095. Both pairs were selected because a Generic/Native reversal was already known. They are diagnostic cases rather than a random sample of models or cuts.

Three separate series. The RegNet factorial experiment contains 40 fresh processes in five complete blocks. An initial ResNet experiment contains 20 fresh Native processes in five four-cell blocks. A later full ResNet replication contains another 40 fresh processes in five eight-cell blocks. Every cell in the full ResNet replication, including Native, was measured anew. It does not append Generic measurements to the earlier Native blocks. All three series remain separate. No process is pooled across them. All 100 confirmatory processes were admitted without failures, exclusions, or replacements. Eighteen earlier H10 pilot processes are excluded from the throughput estimates.

Intervention and runtime chains. Each full block contains both cuts under Generic and Native, each with its productive schedule and with globally at most one outstanding task. The eight cells are run sequentially in randomized order within the block, with a fresh worker process for every cell. Generic's productive schedule uses two workers and a queue limit of two. Native uses eight asynchronous Hailo slots and a FIFO limit of three. In the single-task condition, a global completion gate holds admission ownership through materialization, synchronized TensorRT execution, and Top1/Top5 processing. Native also uses one asynchronous slot. Reducing only the slot count would not impose this global endpoint constraint.

The comparison changes concurrency and its required scheduling policy. Generic and Native still use their respective backend, SDK, handoff, conversion, and materialization paths. These factors are not independently switched. The experiment therefore tests scheduling within the actual runtime chains, rather than an isolated copy or conversion cost.

Artifacts and timing. The experiments retain the exact compiled HEFs, TensorRT engines, precision settings, and prepared input bytes of the deployment cases. They use a frozen copy of the current product runtime sources. This is not a reconstruction of historical software conditions. Every process performs 30 s warmup followed by a complete drain. It then admits work for 60 s and completes all admitted tasks. Throughput divides completed tasks by the entire measured makespan, including fill and drain. Warmup completions are excluded. The timed classification endpoint includes materialized outputs and Top1/Top5 processing. The full ResNet makespans range from 60.00138 to 60.02365 s. No energy measurements were added.

Clock, power profile, affinity, temperature, competing work, and file bindings are checked before and after each run. The fixed settings are MAXN_SUPER, 1.984 GHz CPU, 1.173 GHz GPU, and CPU affinity 0–7. The starting CPU temperature must be 40–65 °C with at most 2 °C drift over 2 s and no active throttle alert. The protocol permits at least 5 s rest and at most 120 s waiting for state admission. End temperatures are descriptive rather than a post-hoc exclusion rule. The full ResNet series reaches end temperatures up to 66 °C.

Functional checks. Sixteen separate H10 pilot processes each exercise 32 mixed requests from two distinguishable prepared inputs. Outputs match byte for byte across runtime chains and scheduling conditions and match the saved functional references. The eight exact ResNet checks are reused for the full replication because sources, artifacts, inputs, and execution paths remain bound to the same configurations. They cover 256 requests and do not enter throughput estimates. These finite checks are not a new 5,000-image quality evaluation or a test of arbitrary input streams. There is no direct measured global high-water counter. The one-task constraint is established by the pinned completion-gate ordering, available queue and inflight counters, and functional checks.

I.1 Contrasts and uncertainty

Within block b, let $r_{v,s,b}$ be the throughput of the later cut divided by that of the earlier cut, for runtime $v \in \{G, N\}$ and schedule $s \in \{P, S\}$. The later cuts are b123 for RegNet and b095 for ResNet. The scheduling contrast for runtime v is $r_{v,P,b}/r_{v,S,b}$. The interaction is

$$I_b = \frac{r_{N,P,b}/r_{N,S,b}}{r_{G,P,b}/r_{G,S,b}}. \tag{5}$$

We average each contrast on the log scale over the five blocks and then exponentiate. For log contrast ℓ_b, the interval is

$$\exp\left(\bar{\ell} \pm t_{0.975,4}\frac{s_\ell}{\sqrt{5}}\right), \tag{6}$$

where s_ℓ is the sample standard deviation and $t_{0.975,4} = 2.776445$. The interaction is a ratio of cut ratios, not a speedup for an individual cut. Blocks are the replication units. Requests within one process are not additional independent replications. The intervals are nominal, with five blocks and no simultaneous coverage correction.

The RegNet protocol allowed a separate five-block extension if any of the log intervals for productive Generic, productive Native, or the interaction contained zero. None did, so no extension processes ran. This conditional design does not establish simultaneous or sequential 95% coverage. The full ResNet replication has a separately fixed five-block protocol and no outcome-dependent extension. Its protocol and analyzer were frozen before the first new process. In the original RegNet study, the mathematical rules were fixed before measurement, while the additional implementation-hash record was created after the first process began and before the first complete block.

I.2 Interpretation and limits

With productive scheduling, Generic prefers the earlier cut and Native the later cut in both full experiments. Under the single-task intervention, both chains slightly prefer b123 for RegNet, while both prefer b002 for ResNet. The intervention reverses Generic's ordering in the RegNet pair and Native's ordering in the ResNet pair. It does not reverse both chains in either experiment. The relative throughput changes in all four runtime/model comparisons.

The interaction is 5.108 for RegNet and 3.429 for the full ResNet replication. Their direction agrees in every block.

Table 17. Complete contrasts from the separate 40-process RegNet and 40-process full ResNet experiments. Values are geometric means with nominal 95% intervals over five complete blocks. SD is the sample standard deviation of block log contrasts. RegNet ratios use b123/b052 and ResNet ratios use b095/b002.

	RegNet, 40 processes		ResNet full, 40 new processes	
Contrast	Mean [interval]	Log SD	Mean [interval]	Log SD
Generic productive, $r_{G,P}$	0.6603 [0.6559, 0.6648]	0.00541	0.6057 [0.6033, 0.6082]	0.00328
Native productive, $r_{N,P}$	3.3713 [3.3555, 3.3872]	0.00378	2.0597 [2.0281, 2.0919]	0.01248
Generic single-task, $r_{G,S}$	1.0289 [1.0228, 1.0350]	0.00478	0.8359 [0.8276, 0.8442]	0.00804
Native single-task, $r_{N,S}$	1.0283 [1.0163, 1.0405]	0.00947	0.8290 [0.8240, 0.8339]	0.00481
Generic scheduling, $r_{G,P}/r_{G,S}$	0.6418 [0.6350, 0.6487]	0.00856	0.7247 [0.7183, 0.7310]	0.00708
Native scheduling, $r_{N,P}/r_{N,S}$	3.2785 [3.2295, 3.3282]	0.01213	2.4847 [2.4382, 2.5320]	0.01520
Interaction, I	5.1081 [4.9896, 5.2294]	0.01890	3.4288 [3.3532, 3.5061]	0.01796

Table 18. Blockwise contrasts in the two full experiments. Each row contains eight independent worker processes measured in randomized order. Scheduling columns divide the productive cut ratio by the single-task cut ratio. The final column divides the Native scheduling contrast by the Generic scheduling contrast.

		Productive r		Single-task r		Scheduling contrast		
Series	Block	Generic	Native	Generic	Native	Generic	Native	I
RegNet	1	0.65765	3.36887	1.02336	1.03003	0.64264	3.27064	5.08936
RegNet	2	0.66320	3.38481	1.02897	1.03296	0.64452	3.27681	5.08408
RegNet	3	0.65670	3.37013	1.03603	1.02491	0.63386	3.28822	5.18760
RegNet	4	0.66496	3.35211	1.02540	1.03995	0.64849	3.22334	4.97053
RegNet	5	0.65928	3.38085	1.03061	1.01393	0.63970	3.33440	5.21247
ResNet full	1	0.60584	2.01615	0.83901	0.83102	0.72209	2.42613	3.35988
ResNet full	2	0.60686	2.06184	0.83123	0.82767	0.73008	2.49113	3.41214
ResNet full	3	0.60280	2.07828	0.83492	0.82749	0.72198	2.51155	3.47871
ResNet full	4	0.60805	2.07796	0.84567	0.82412	0.71901	2.52144	3.50682
ResNet full	5	0.60502	2.06515	0.82861	0.83464	0.73017	2.47431	3.38869

Table 19. Earlier Native-only ResNet series, retained separately. These 20 processes are not part of the full replication. Values use five four-cell blocks and b095/b002.

Contrast	Mean [nominal 95% interval]
Productive	2.0542 [2.0467, 2.0617]
Single-task	0.8310 [0.8284, 0.8336]
Scheduling	2.4720 [2.4611, 2.4829]

Block	Productive	Single-task	Scheduling
1	2.05410	0.83111	2.47153
2	2.04726	0.83206	2.46048
3	2.05262	0.82736	2.48092
4	2.06387	0.83209	2.48035
5	2.05314	0.83235	2.46667

Their differing magnitudes are descriptive. The study tests neither equal effect sizes nor a formal difference between models. The selected cases demonstrate a scheduling-dependent ordering in these runtime chains. They do not estimate its prevalence across unknown models, devices, or cuts, and they do not identify a single copy, layout, conversion, or historical software cause.

Supporting Native counters. The original RegNet productive runs have median total cycle intervals of 2.243 ms at b052 and 0.666 ms at b123. Their measured serial consumer intervals are 2.218 and 0.613 ms, accounting for 98.98% and 92.29% of measured duration. Synchronized TensorRT work, including H2D, execution, D2H, and stream synchronization, falls from 2.020 to 0.451 ms. Median FIFO put blocking is 1.675 versus 0.021 ms, and queue waiting is 8.366 versus 0.322 ms. These counters support a stronger consumer bottleneck at b052. They do not establish that b123 is exclusively prefix-limited.

In the original Native-only ResNet series, the productive consumer accounts for 98.71% of duration at b002 and 89.98% at b095. FIFO put blocking is 1.445 versus 0.0126 ms. Under single-task execution, total cycle intervals are 5.123 and 6.155 ms. These values are retained as observations of that separate series. Pipeline sections overlap, queue waiting includes put blocking, and the single-task prefix response includes waiting for global admission. Their times must not be added as independent costs. Corresponding Generic stage counters are unavailable rather than zero.

The separate stage-cost experiment in Section K uses RegNet b059/b123, not this experiment's b052/b123 pair. Its

independent stage measurements test a composition rule. They are not a controlled counterfactual for the scheduling experiment or a substitute for its full-factor contrasts.

J Graph-Score Comparison and Feasibility

This experiment compares five static scores and Generic against a fixed set of Native reference measurements (Mika, 2026). It contains 204 technical split pairs and 42 Full references. Energy measurements are available for 195 splits and 36 Fulls over six models, excluding YOLOv7. These are different runtime measurements from the deployment comparison, whose seven-model energy coverage is complete.

When selecting one cut, Generic finds 11 of 21 Native winners with 12.74% mean loss in the graph-score comparison. None of the five static selectors finds a classifier winner with its first choice. The deployment comparison instead yields nine matches and 15.16% mean loss. The difference reflects distinct Native reference measurements, not a change in the loss formula. The full-budget analysis below tests whether these rankings become useful when more candidates can be measured.

J.1 Selector definitions

Candidate graph. The existing tool enumerates boundaries along a deterministic topological ordering of the ONNX DAG. These are prefix cuts, not arbitrary graph partitions. The frontier retains every tensor produced in the prefix and consumed in the suffix, with deduplication when one tensor has several suffix consumers. Original inputs required directly by the suffix are tracked separately. Constants remain initializers. A candidate can be graph-valid yet fail native input-count or vendor-compiler restrictions.

The seven model universes contain 138, 133, 120, 615, 416, 382, and 307 boundaries for MobileNetV3-Large, RegNetX-1.6GF, ResNet-50, YOLO11l, YOLO26m, YOLO26s, and YOLOv7, respectively. The 2,111 boundaries are distinct from the 204 model/boundary/setup observations. The analysis does not claim to profile every graph boundary.

Formulas and profiles. The weighted selector uses weights $(1, 3, 0.2)$ on communication, imbalance, and excess tensor count. Communication is transformed with $\log_{10}(1 + b)$. Excess count is $\max(k - 1, 0)$. Each term is min/max-normalized in the complete model universe before subset or quality filtering. Static ties use boundary index followed by case identity. Measured Generic ties preserve original pair order.

The cycle heuristic uses the numerical estimate

$$\widehat{T}_{\text{total}} = 5 + \frac{F}{10^{10}} + \frac{b}{10^{9}}, \tag{7}$$

then distributes it by the larger recorded compute share to obtain a bottleneck estimate. This is an empirical graph heuristic. Although the software labels its output in milliseconds, it is not a dimensionally calibrated time model. Its graph-operation estimate F is not a measured processor-operation count. No device-specific stage-time model or throughput fit is configured. The streaming heuristic inverts the bottleneck plus its heuristic handover estimate. No calibrated Native-handover model is included.

All accelerator-fit profiles use Hailo10 for the first stage and TensorRT for the second. Their low/ideal/high/hard-high compute-share parameters are 0.25/0.42/0.58/0.72. Cut thresholds are 10/20 and over/under weights 2.2/0.6. The original implementation supplies decimal megabytes to parameters whose names contain "MiB". The analysis reproduces that behavior and does not claim a corrected selector evaluation.

No calibrated device specification was configured for the total-latency or Native-handover estimators. Those estimators are therefore not evaluated. No score in this separate graph-score experiment is fitted to the Native outcomes. Across the five available static methods, only 33 of 105 global recommendations have completed measurements. The global winner is never replaced by a convenient measured candidate.

The repeat-sensitivity ranges enumerate nine dependent Generic/Native repeat combinations per group. They are not confidence intervals or nine new experiments. Uniform random Top1 over those finite sets has 4.484 expected hits, of which four come from detector groups. The Generic Top3 shortlist contains six of nine classification winners in this experiment. The deployment comparison contains two of nine. The counts apply to their respective Native reference measurements.

The weighted and no-handover cycle rules select the same candidate in all 21 groups, although their full rankings differ. Their matching choices are not independent evidence for the same cost model.

The quality-tolerance sensitivity fixes the fastest local Full before applying 1%, 2%, and 5% numerical relative-loss thresholds plus its recorded semantic requirements. It retains 105, 136, and 178 of 195 energy-covered split cases, respectively. The corresponding joint-throughput/energy benefits are 31, 51, and 67. These are deterministic threshold comparisons within this experiment, not additional measurements or deployment-comparison counts.

Table 20. Graph-score comparison on 21 fixed model/setup groups. Classification groups contain 17–20 candidates and detection groups contain three. All methods use the same Native reference measurements. Energy excess is evaluated in the 18 groups with matching energy measurements, excluding YOLOv7. Each group has equal weight.

	Native Top1 hits			Throughput loss L_R (%)		Mean energy
Selector	All /21	Class. /9	Detect. /12	Median	Mean	excess /18 (%)
Smallest boundary bytes (BYTES)	6	0	6	3.77	25.28	55.09
Weighted graph score (WEIGHTED)	6	0	6	13.79	17.28	24.57
H10 accelerator-fit score (FIT-H10)	6	0	6	12.81	19.65	32.02
Heuristic cycle without handover (CYCLE)	6	0	6	13.79	17.28	24.57
Streaming-rate heuristic (STREAM)	7	0	7	9.25	17.42	24.57
Measured GENERIC completion	11	3	8	0.00	12.74	28.07

Table 21. Graph inventory and completed Native splits per setup. The graph-cut count includes multi-tensor frontiers. Native counts are the measured supported subset, including the additional detector candidates.

Model	Graph cuts	Native splits/setup
MobileNetV3-Large	138	17
RegNetX-1.6GF	133	20
ResNet-50	120	19
YOLO11l	615	3
YOLO26m	416	3
YOLO26s	382	3
YOLOv7	307	3
Total	2,111	68

J.2 Complete rankings and shortlist curves

Figure 6 extends the comparison to every measured candidate and every shortlist size. Its classifier curves use all 168 cuts in nine groups of 17–20 candidates. Scores, parameters, and tie rules remain fixed. Spearman correlation compares each complete score ranking with the Native ranking, using average ranks for ties. A positive value means that better predicted cuts tend to have higher measured Native throughput.

The boundary-size rule has a mean classifier rank correlation of 0.069 and selects no exact classifier winner at one trial. Its larger shortlists perform better. At $k = 11$, every group contains a cut within 5% of its best Native throughput. At $k = 12$, eight groups contain an exact winner, and at $k = 13$ all nine do. Weighted and Fit-H10 need $k = 19$ for all nine exact winners. Cycle, Stream, and Generic need $k = 18$ (Table 16). The boundary-size rule therefore has useful shortlist behavior despite its weak first choice and uneven full-rank correlation.

The twelve detector groups are evaluated separately because each contains only three candidates. Weighted, Fit-H10, Cycle, Stream, and Generic recover every exact detector winner with two trials. Boundary size needs three. The results cover the five available tool rules and measured Generic completion. They do not supply predictions for the unconfigured estimators above or for the original CNNParted, Timeloop, or Accelergy methods.

Table 22. Independent stage-cost experiment on four known cuts. Median steady-state producer, consumer, and pipeline rates (completions/s) over three blocks. Prediction is $\min(F_P, F_C)$. Error is the median of the blockwise absolute percentage errors.

Model/cut	F_P	F_C	F_{PC}	Error (%)
ResNet/DX b045	230	669	225	2.24
ResNet/DX b002	473	520	444	5.50
RegNet/H10 b059	1565	500	477	4.77
RegNet/H10 b123	1577	1999	1518	3.85

K INDEPENDENT STAGE-COST EXPERIMENT

The stage-cost experiment evaluates ResNet-50/DeepX b045 and b002, and RegNet-X/H10 b059 and b123. Three blocks per case contain separate producer (P), consumer (C), and pipeline (PC) processes. There are 36 performance processes and twelve blockwise stage-composition comparisons. Four serial-reference processes and twelve smokes are additional preparation, not performance repetitions.

Each process uses batch one, 100 fully drained warmups, and 10,000 measured completions. The reported core rate is $8000/(t_{9000} - t_{1000})$, with times in seconds and one-based observed completion order. The term core denotes this fixed middle window, not a hardware core. Total rates retain fill and drain. The halves t_{1000}–t_{5000} and t_{5000}–t_{9000} are retained for the predefined 5% drift check. No faster interval is chosen after measurement.

P ends when a usable host-side boundary tensor is available. DeepX uses its synchronous producer in this experiment. H10 uses its stored eight-request producer. Both include output mapping and the required ownership transfer. C rotates four genuine saved boundary buffers and includes host staging, H2D, TensorRT execution, D2H/synchronization, and ordered Top1/Top5. PC uses the same producer and consumer functions with FIFO depth three. Per-task copies

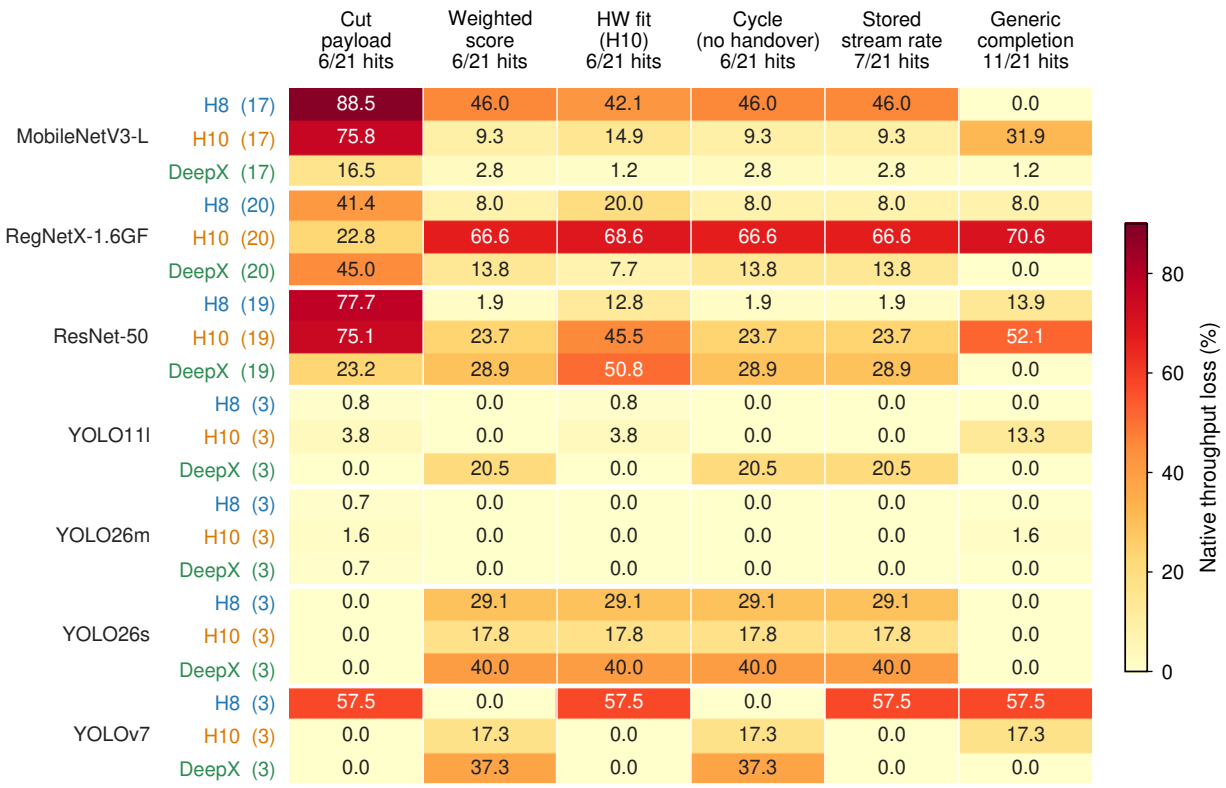


Figure 5. Single-choice throughput loss in the graph-score comparison. Each column is a selector and each row a model/setup group. All columns use the same Native reference measurements. The H10-fit profile is shared across setups.

remain timed. Replayed consumer buffers and fresh PC payloads have different cache and allocation behavior, which limits the interpretation of the composition residual.

Within each case and block, the prediction

$$\widehat{F}_{PC} = \min(F_P, F_C) \tag{8}$$

is formed before the PC report is read. The signed error is $100(\widehat{F}_{PC}/F_{PC} - 1)$. The cycle residual is $1000/F_{PC} - \max(1000/F_P, 1000/F_C)$ in milliseconds. Effective intervals are not request latencies. The manuscript checks all twelve block calculations against the original compact exports before aggregating them. A median error need not equal the ratio of the displayed median rates.

All twelve estimates are optimistic. Per-case median errors are 2.24%, 5.50%, 4.77%, and 3.85% in the case order above. The largest individual error is 8.80%. Both within-model orderings agree in all three blocks, giving six matching ordering checks. The 10% band was a predefined practical criterion, not a guaranteed accuracy bound. These results test a composition rule with measured inputs. They do not validate CNNParted, Timeloop, or Accelergy estimators, an inexpensive predictor for unseen cuts, measurement-cost savings, or an energy model.

The recipes repeat one prepared input. Reference parity and pre/post checks do not replace dataset-quality evaluation. Integer payloads are checked exactly. Float payloads use the recorded $\mathrm{rtol} = 10^{-4}$ and $\mathrm{atol} = 10^{-5}$, with matching ordered Top1/Top5. Clock, affinity, temperature, and missing-EMC observations remain attached to the original reports. The recorded settings are MAXN_SUPER, 1.984 GHz CPU and 1.173 GHz GPU clocks. These records do not establish unobserved device occupancy or isolate thermal, DMA, memory, or scheduler causes.

The summary and all twelve block comparisons are provided with the analysis. Predictions use only the independently measured stages of this experiment, not rates from the deployment comparison (Mika, 2026).

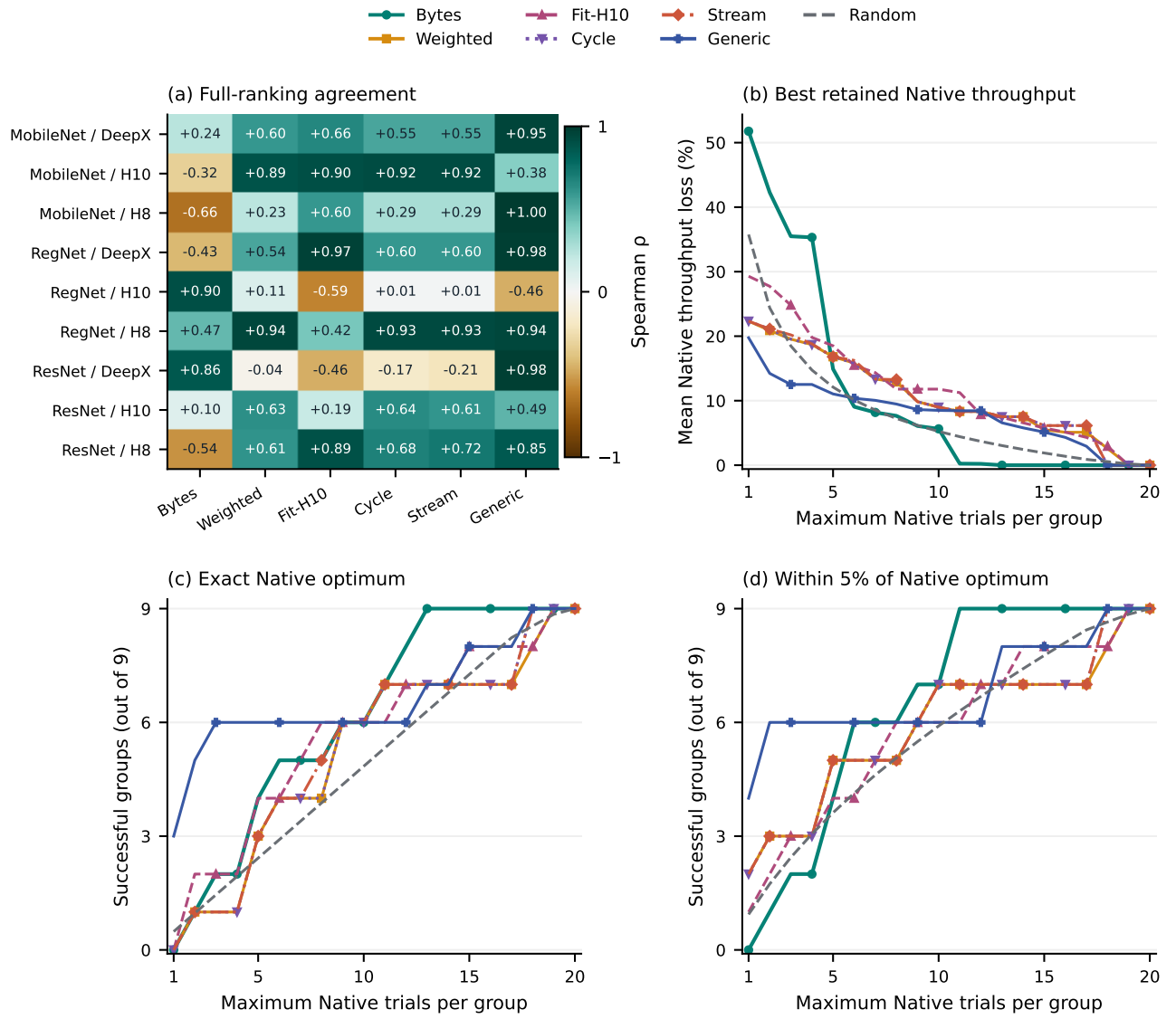


Figure 6. Complete graph-rule comparison on the fixed classifier measurement set. The panels show full-rank correlations, mean Native throughput loss, recovery of an exact Native winner, and recovery of a cut within 5% of the Native optimum. All curves retain the same nine model/setup groups. Random curves show exact finite-set expectations. The 5% tolerance concerns throughput, not dataset quality.

Table 23. Splitting can recover task quality lost in a full accelerator deployment. MobileNetV3 Top1 on the same 5,000 images. CPU ONNX reference: 73.70%. The split uses b027 and a TensorRT suffix.

Configuration	H8	H10	DX
Accelerator Full (%)	60.62	59.42	71.72
Split b027 (%)	71.58	72.22	72.18
Quality recovered (pp)	10.96	12.80	0.46

L Compiled Feasibility and Task Quality

The compiler must accept the actual subgraph and its tensor shapes. In the YOLO11l compilation tests, some boundaries leave successor relationships or output shapes ambiguous. Others expose detection-head `Reshape`/`Transpose` sequences that cannot be mapped. An H8 mapping fails when an internal format-conversion layer cannot be assigned resources. Operator names alone therefore do not determine buildability (Mika, 2026).

All 21 TensorRT Fulls meet the 5% point-estimate relative-quality-loss rule, while nine of 21 accelerator Fulls exceed it. H10 MobileNet Full loses 14.28 percentage points of Top1 accuracy. RegNet and ResNet on that setup remain within 0.16 points of their float references. A feasibility estimate cannot replace dataset evaluation.

Partitioning can reduce a Full model's quality loss (Table 23). The MobileNet b027 splits use the same 5,000-image evaluation and a 73.70% CPU reference. These are outcomes of complete compiled recipes. They do not isolate quantization from calibration, graph transformation, or boundary handling.

M Compiler and Tensor-Interface Experiments

These experiments examine compiler and tensor-interface effects on specific artifacts. Their images and timed samples are separate from the deployment and selector comparisons. The original measurements are provided under `analysis/vendor_sources/`.

Host input representation: a matched Hailo-10H probe. For one YOLO26s b024 HEF under HailoRT 5.3.0, a deterministic UINT8 tensor q defines the device input. The FLOAT32 arm receives its dequantization. HailoRT's transformation is checked to recover q exactly before inference. Both arms expose native UINT8 outputs and permit only one outstanding job. Over 100 timed samples per representation, mean latency is 49.62 ms for FLOAT32 input and 12.77 ms for UINT8 input (3.89×). These are within-run samples, not 100 independent process repetitions. The compared 819,200-element output buffers match exactly. Separately timed external quantization costs 1.26 ms, giving 14.03 ms when added to the native-input latency. These stage measurements neither include a TensorRT tail nor establish task accuracy or energy savings. Source: `h10_input_ab.json`.

Output format separates latency from throughput. A companion probe retains FLOAT32 input and changes only host output representation. Native output reduces mean single-job latency from 63.03 to 49.74 ms and output-buffer size fourfold. Nevertheless, with two outstanding jobs, observed throughput is 28.35 versus 28.29 FPS. External dequantization is measured separately at 0.36 ms. The compared dequantized values match exactly. This illustrates why a stage-latency improvement need not translate into higher pipelined throughput. Source: `h10_output_ab.json`.

A scalar output scale can erase detector scores. A stored-output replay examines one image each for Hailo-10H YOLO26m b398 and YOLO26s b364. Their physical UINT8 outputs have shape $[1, 8400, 84]$. All 672,000 score entries per model equal the respective zero point (39 or 35), while coordinate codes vary. A common scale of 3.4920754433 or 3.1612741947 covers both coordinates and probabilities: scores at most one fall below half a quantization step. Quantizing the corresponding float scores reproduces their collapse. The correctly bound dequantization/layout bridge cannot restore them. Unchanged decoding yields zero detections, versus five and eight from the float references. The first observed loss is at or before the compiled output contract, not newly introduced by host mapping. Internal prequantization activations are unavailable, so no particular compiler pass is isolated. This replay performs four local ONNX Runtime tail evaluations and no new device runs. Source: `h10_score_contract.md`.

Compiler stages are not interchangeable accuracy references. On 16 Hailo-8 MobileNet images, source ONNX, compiler ONNX, and parsed-HAR each classify 13 correctly. Optimized/quantized HAR classifies nine correctly, changing five Top-1 predictions. Its associated GPU-build HEF classifies ten correctly and differs from emulation on four predictions. The first large deviation spans optimization and quantization jointly. It does not isolate rounding or explain all main-campaign losses. Sources: `h8_har_comparison.json` and `h8_har_report.md`.

Toolchain documentation. The vendor documentation describes graph optimization, quantization, supported operators, and tensor interfaces (Hailo, n.d.; DEEPX, n.d.a;n; NVIDIA, n.d.b;n). These documents identify the intended contracts. They do not replace the artifact-specific measurements above.

N Energy Calibration and Measurement Windows

Primary energy covers the DC system input, including host and attached accelerator and excluding upstream AC/DC losses. The original setup identifies an input shunt and INA225 acquisition path. Stored setup-calibration gains are 0.913634 for H8, 0.896397 for H10, and 0.903508 for DeepX, using the retained original load/idle fits. A common multiplicative calibration cancels in a same-setup energy ratio but not, in general, in a comparison across setups. Nonlinear or load-dependent errors need not cancel.

The H10 calibration retains warnings for a 2.026% point-factor spread and a gain below a configured 0.90 bound. The calibration captures use 2 kS/s and 30 s intervals. These calibration settings do not specify the acquisition settings of every experiment or provide a complete uncertainty budget. The original calibrations remain under `analysis/review_inputs/calibrations/`, with the existing summary under `analysis/review_output/`.

The command-window records independently establish 2 kS/s acquisition for all 225 energy repetitions of the 75 DeepX Native configurations. Their compact projection and source hashes are archived under `analysis/r20_evidence/`. This rate is specific to those recorded workload windows.

Longer-window analysis. The separate energy-window study contains 693 repetitions with 61.28–63.10 s completion windows. Its plots use N_r/T_r from those same windows, rather than an independently measured performance median. In that six-model study, 16 of 18 quality-filtered model/setup groups retain the same rate winner between the shorter performance and energy-window experiments. The two changes are YOLO11l/H10 b120 to b062 and YOLO26m/DeepX b043 to b038. Its ResNet/H8 b031 and b060 example sacrifices 1.79% energy-window rate for 11.38% lower energy. The deployment comparison instead selects b038 and b060 using its separate performance measurements. Neither study's window test certifies indefinite thermal stability.

For YOLO11l, b062 reaches 37 FPS at 641.55 mJ/image in those longer windows. Reselecting the fastest quality-admissible Full by the same window rate gives TensorRT on H8 at 36 FPS and 707.36 mJ/image. The split is 4.4% faster and uses 9.3% less energy, calculated before rounding. These earlier runtime paths do not include the current DeepX Full implementation. This comparison therefore does not repeat the main Table 3 over a longer window. In the current deployment data, b062 instead reaches 51 FPS at 558.64 mJ/image, while DeepX Full reaches 63 FPS at 315.23 mJ/image.

O Software and Reproduction

The ONNX-Splitpoint-Tool performs graph analysis, subgraph export, and the supported hardware workflows. It was developed with large-language-model coding assistance under author-defined requirements. Generated implementations were evaluated using regression tests, numerical output checks, and hardware runs. This development method is not itself an experiment in programming productivity. The authors remain responsible for the software, experimental design, and interpretation.

Hardware execution and numerical reproduction are separate. The tool invokes vendor compilers and runtimes to obtain measurements. The accompanying offline analysis uses compact measurement exports to reconstruct the tables, selections, and figures. It does not need accelerator hardware or regenerate measurements.

The original publication profile and its supporting evidence are pinned to commit `b5c7c59` in the accompanying repository (Mika, 2026). The full-budget analysis uses the recorded inputs at commit `b10d159`. Its scripts and outputs are published at commit `9a7d375`:

```
https://github.com/Keff789/edge-ai-performance-energy-results
```

The MLSys entry page links the complete supplement, its LaTeX sources, and the original `EVIDENCE_AUDIT.csv` and `FLAG_RESOLUTION.csv` records for all 450 configurations. The scheduling evidence packages contain the original process reports, frozen runtime sources, and recorded device states. Their manifests preserve the original archive and member hashes. The two forty-process studies and the earlier twenty-process Native-only ResNet series remain separate.

The `mlsys-r22` publication profile regenerates the numerical figures and tables from the recorded inputs. Its asset manifest links each output to its source files and generator. The additional public records support inspection of the audit decisions and measurement provenance. Models and accelerator hardware are not required for the offline analysis.

From the repository root, run:

```
python3.12 -I -B \
scripts/split-inference/reproduce.py \
--output /tmp/edge-ai-reproduced \
--publication mlsys-r22 --verify
```

The repository documents the pinned environment and separate study commands. Adding `--numeric-only` reproduces numerical outputs and TeX tables without rendering figures.

The full-budget selection analysis has a separate entry point and pinned requirements file in its script directory. It was reproduced with Python 3.12.14:

```
python3.12 -I -B \
scripts/supplementary-studies/\
rq2-all-cuts-20261020/reproduce.py \
--output /tmp/edge-ai-rq2-all-cuts \
--verify
```

Its published outputs are under `results/supplementary-studies/rq2-all-cuts-20261020/`. They include complete candidate rankings, group-level loss and success curves, first successful ranks, actual trial counts, and minimum budgets for each target. The scripts retain the separate graph-score and deployment measurements. The publication figures and budget table are generated from these numerical outputs. Adding `--numeric-only` omits figure rendering. No new hardware measurements or calibration fits are required.

Within the manuscript package, `make all` compiles both documents from the supplied tables and PDF figures. Numerical reproduction uses the accompanying repository and the commands above. These commands do not run hardware measurements, model compilation, or dataset evaluation.

Source limitations. The audit in Section G resolves the earlier administrative restrictions for the documented deployment comparison. It does not reconstruct every historical software dependency or establish equal schedules across backends. Distinct-input checks cover eight exactly bound historical DeepX configurations, while the new scheduling tests have their own narrower functional checks. Repeated prepared inputs do not establish arbitrary request/output association. Original records and review decisions remain available.

REFERENCES

DEEPX. Execution of DX-COM. DXNN SDK 2.4.0 documentation, n.d.a. URL https://developer.deepx.ai/tech-docs/2.4.0/DXNN-SDK/DX-COM/02_06_Execution_of_DX-COM/. Accessed October 5, 2026.

DEEPX. Supported ONNX operators. DXNN SDK 2.4.0 documentation, n.d.b. URL https://developer.deepx.ai/tech-docs/2.4.0/DXNN-SDK/DX-COM/Building_Models/. Accessed October 5, 2026.

Hailo. Hailo Model Zoo: Model optimization. Official software documentation, n.d. URL https://github.com/hailo-ai/hailo_model_zoo/blob/master/docs/OPTIMIZATION.rst. Accessed October 5, 2026.

Mika, K. Edge AI performance–energy results. Versioned research artifact, 2026. URL https://github.com/Keff789/edge-ai-performance-energy-results/tree/9a7d3754691f0d54646e789f485d1565c87b268a. Source snapshot: commit 9a7d375.

NVIDIA. TensorRT operators. Official software documentation, n.d.a. URL https://docs.nvidia.com/deeplearning/tensorrt/latest/api/operators.html. Accessed October 5, 2026.

NVIDIA. Working with quantized types. TensorRT 10.x documentation, n.d.b. URL https://docs.nvidia.com/deeplearning/tensorrt/10.x.x/inference-library/work-quantized-types.html. Accessed October 5, 2026.